PS1 Python 作业报告

1.Flowchart：三个数的排序

(1)代码要做什么

Print_values(a, b, c) 接收三个数，通过比较大小决定输出顺序。目标是让最大的数出现在最前面，最小的数出现在最后。函数直接使用 print 输出，没有用 return 返回一个排序列表。

(2)判断过程

先比较 a 和 b。如果 a > b，再看 b 是否大于 c；如果成立，顺序就是 a、b、c。如果 b 不大于 c，就继续比较 a 和 c，决定 c 应该放在中间还是最前面。如果 a 不大于 b，就进入外层 else。此时再比较 a 和 c：若 a > c，输出 b、a、c；否则再比较 b 和 c，分别输出 b、c、a 或 c、b、a。每一条分支都把三个数安排到对应位置。

```
if a > b:
    if b > c:
        print(a, b, c)
    elif a > c:
        print(a, c, b)
    else:
        print(c, a, b)
```

(3)保存的运行结果

代码测试了 (9, 6, 3)、(3, 9, 6)、(3, 6, 9) 和 (6, 9, 3)。四次都输出 9 6 3，说明这四种输入顺序得到了相同的降序结果。原来的四次测试没有覆盖所有排列，也没有测试相等的数。本次额外检查了由 0、1、2 组成的全部 27 个三元组，输出均与降序结果一致。这个检查覆盖了重复值，但不等于验证所有输入类型。

2. Matrix multiplication：矩阵乘法

主要方法生成了一个 5 × 10 的矩阵 M1 和一个 10 × 5 的矩阵 M2。每个元素用 random.randint(0, 50) 生成，包括 0 和 50。矩阵用嵌套列表保存：外层列表存放各行，内层列表存放一行中的元素。乘法部分使用 for 循环、乘法和加法完成，没有调用现成矩阵乘法函数。

(1)为什么结果是 5 × 5

矩阵乘法要求前一个矩阵的列数等于后一个矩阵的行数。这里两个数量都是 10，因此可以相乘。乘积保留 M1 的行数和 M2 的列数，所以结果有 5 行、5 列。结果中第 i 行、第 j 列的元素，是 M1 的第 i 行与 M2 的第 j 列对应元素相乘后相加。代码里的下标从 0 开始，所以 M1[0][0] 表示第 1 行第 1 列。

```
for i in range(5):
    row = []
    for j in range(5):
        total = 0
        for k in range(10):
            total = total + M1[i][k] * M2[k][j]
        row.append(total)
    result.append(row)
```

i 选择结果的行，j 选择结果的列，k 遍历需要相乘的 10 对元素。total 必须在每个新元素开始计算时归零，否则上一个元素的结果会被带入。row 在一行开始时设为空列表，用来收集这一行的 5 个结果。

(2)结果与核对

```
[5399, 3892, 4981, 6321, 7065]
[4687, 2830, 3552, 3900, 5427]
[4669, 3263, 3214, 6124, 5475]
[5974, 5002, 4716, 5827, 5490]
[5094, 3644, 5447, 7895, 5794]
```

例如左上角是 35×8 + 42×18 + 19×6 + 40×6 + 18×7 + 3×42 + 36×33 + 9×41 + 50×40 + 8×25 = 5399。本次使用保存的两个输入矩阵重新核对，25 个结果都一致。

补充方法生成是那天上完课得出的。

3. Pascal triangle：杨辉三角

(1)第一种方法：从上一行生成下一行

程序把 [1] 作为第 1 行。每次生成新的一行时，先放入开头的 1，再把上一行相邻的两个元素相加，最后补上末尾的 1。例如从 [1, 2, 1] 得到 [1, 3, 3, 1]。

```
row = [1]
for i in range(1, k):
    new_row = [1]
    for j in range(len(row) - 1):
        new_row.append(row[j] + row[j + 1])
    new_row.append(1)
    row = new_row
```

range(1, k) 一共执行 k−1 次，因为最初已经有了第 1 行。row = new_row 表示本轮得到的新行会作为下一轮的输入。程序没有保存整个三角形，只保留当前的一行。

(2)第二种方法：用组合数直接计算

```
from math import comb
row = [int(comb(k - 1, i)) for i in range(k)]
```

按“[1] 是第 1 行”的约定，第 k 行第 i+1 个元素为 C(k−1, i)。列表推导式让 i 从 0 到 k−1，逐个计算后组成列表。comb 已经返回整数，因此外面的 int 可以省略；这个方法也没有实际使用 NumPy。本次逐项比较了两种方法生成的第 100 行和第 200 行，结果完全相同。还可以用每行元素之和等于 2^(k−1) 来检查。Python 的普通整数能保存这些很大的整数，不需要改为浮点数。

原题所链接的 Math is Fun 页面明确把顶端 [1] 标为 Row 0 [2]，因此当前两次调用分别对应该网页的 Row 99 和 Row 199。若按网页编号计算 Row 100 和 Row 200，应分别有 101 和 201 个元素。作业正文使用“第 k 行”的表述，没有另外定义起始编号；报告应明确采用的约定，不能把两种编号混用。当前稿如实保留原代码输出，没有把它改写成另一套编号的结果。另需注意，两个函数同名，后定义的版本会覆盖前一个。

4. Add or double：最少操作次数

按照代码的含义，起点是 1，每一步可以加 1 或乘以 2，目标是正整数 x。Least_moves(x) 从目标倒着算：遇到偶数就除以 2，遇到奇数就减 1，直到回到 1。

```
moves = 0
while x > 1:
    if x % 2 == 0:
        x = x // 2
    else:
        x = x - 1
    moves = moves + 1
```

x % 2 用来判断奇偶，余数为 0 表示偶数。x // 2 是整数除法。moves 每轮增加 1，记录倒推的次数；倒推一步对应正向操作的一步。

(1)为什么可以这样倒推

奇数不能由整数翻倍得到，所以它的最后一步只能是加 1，倒推时减 1 是必需的。对于大于 2 的偶数，如果倒推选择减 1，得到的奇数还要再减 1，两步才减少 2。除以 2 可以更快接近起点。

更具体地说，任何正向方案都可以按翻倍操作分成几段加一。如果某次翻倍之后连续加了两次 1，把这两次加一换成翻倍前加一次 1，最终数值相同，操作却少一次。因此最优方案在每次翻倍后至多加一次 1；偶数目标可以选择以翻倍结束，奇数目标则必须以加一结束。这正好对应代码的倒推规则。本次还核对了 1—100 的最优次数，结果一致。

(2)示例结果

x=2 时只需 1→2，共 1 次。x=5 可以走 1→2→4→5，共 3 次，与保存的输出一致。

保存的随机测试分别为：23 需要 7 次，28 需要 6 次，41 需要 7 次，88 需要 8 次，9 需要 4 次。例如 23 的倒推过程是 23→22→11→10→5→4→2→1，共 7 次。

函数只输出次数，没有输出操作路径，也没有返回 moves。它没有检查输入范围：输入 0 或负数会打印 0，不能理解为正确解决了这类输入。上述分析以 x 是正整数、起点为 1、只允许加 1 和翻倍为前提。

5.1 数字表达式：寻找指定目标

Find_expression(x) 保持数字 1—9 的顺序不变，在相邻数字之间加入加号、减号，或者不加符号。不加符号表示拼接成多位整数，例如 1 和 2 拼接得到 12。这里不使用乘号、除号、括号，也不允许在开头添加负号。

(1)枚举过程

```
for ops in itertools.product(["+", "-", ""], repeat=8):
    expression = digits[0]
    for i in range(8):
        expression = expression + ops[i] + digits[i + 1]
    if eval(expression) == x:
        print(expression, "=", x)
        count = count + 1
```

9 个数字之间有 8 个空位，每个空位有 3 种选择，因此一共有 3^8 = 6561 种表达式。itertools.product 逐个给出这 8 个空位的选择，内层循环负责把符号和数字接成字符串。eval(expression) 计算字符串表示的算式。如果结果等于 x，程序就打印这个表达式，并让 count 增加 1。函数最后返回 count，外面的 print 再显示解的总数。

notebook 的这次随机目标是 2，程序找到了 11 个表达式。下面列出其中 3 个，完整结果在原文件中。

```
1+2+34-5-6-7-8-9 = 2
12+3+4+5+67-89 = 2
123-45+6+7-89 = 2
Total solutions = 11
```

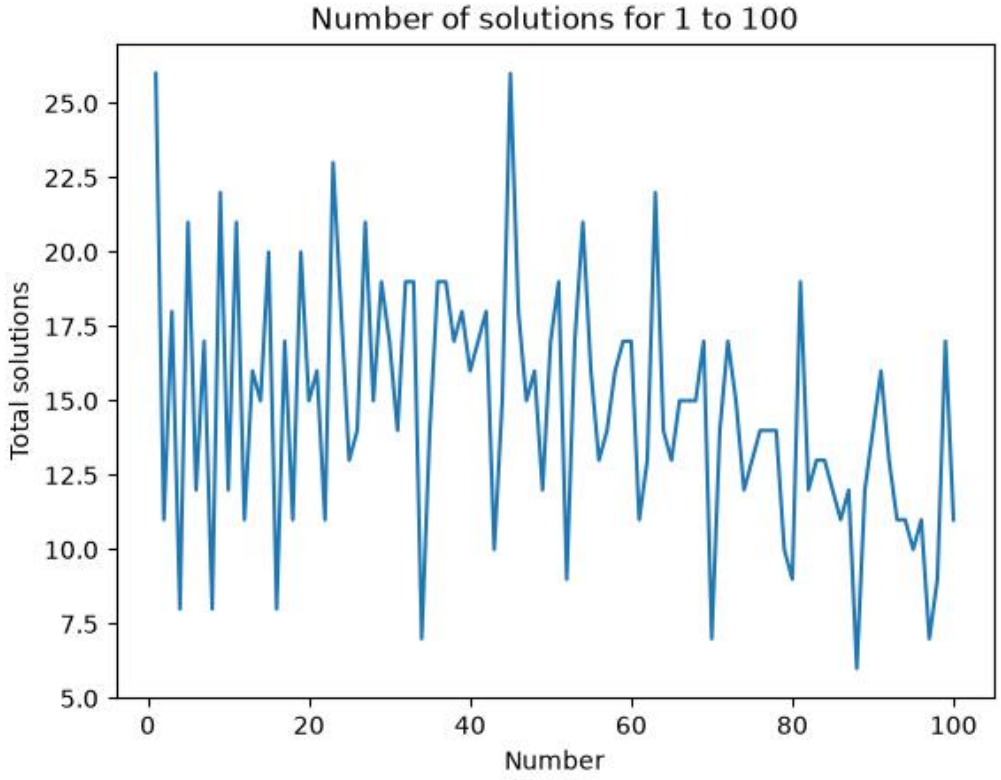
例如最后一个表达式是 123−45+6+7−89，依次计算得到 78、84、91、2。本次重新枚举也得到 11 个解。统计的是符合要求的表达式个数，而不是不同的计算结果个数。

5.2 统计 1—100 的解数与代码检查

第二部分依次取 x=1、2、…、100。对每个目标重新枚举 6561 个表达式，记录满足条件的数量，再把结果放入 Total_solutions。列表第 0 项对应目标 1，所以输出对应目标时需要用 i+1。

(1)数量与图形的含义

横轴是目标值，纵轴是满足条件的表达式数量。曲线表示各个目标的解数差异，不表示程序运行速度。目标增加时解数没有单调增减关系。保存的输出为：最多 26 个解，对应目标 1 和 45；最少 6 个解，对应目标 88。本次独立枚举验证了这些数值。统计范围仅限 1—100，不能把它解释为所有可能结果的全局极值。



图：原 notebook 保存的 1—100 目标值对应解数。

(2)目前的运行问题

最后一行 print(i + 1) 没有缩进到 if 下面。当前源码会在执行该单元格之前报 IndentationError，因此连前面的绘图也不会执行。文件中有保存的图和结果，只能说明 notebook 保留过这些输出，不能证明它们来自当前这份有缩进错误的源码。

此外，第 5.2 部分依赖第 5.1 部分导入的 itertools。重新启动内核后，单独运行第 5.2 部分也会缺少这个名称。